



泛型多态与泛型约束

北京理工大学计算机学院
金旭亮



1 泛型的多态特性

JDK中的“泛型”类型

- 在JDK中，定义了大量的泛型类型（包容泛型类和泛型接口），并且这些类型之间存在着复杂的（基类）继承和（接口）实现关系。
- 基于泛型接口和抽象泛型类，在JDK中大量出现了基于泛型的“多态”代码，把握“泛型多态”特性，是用好它们的前提



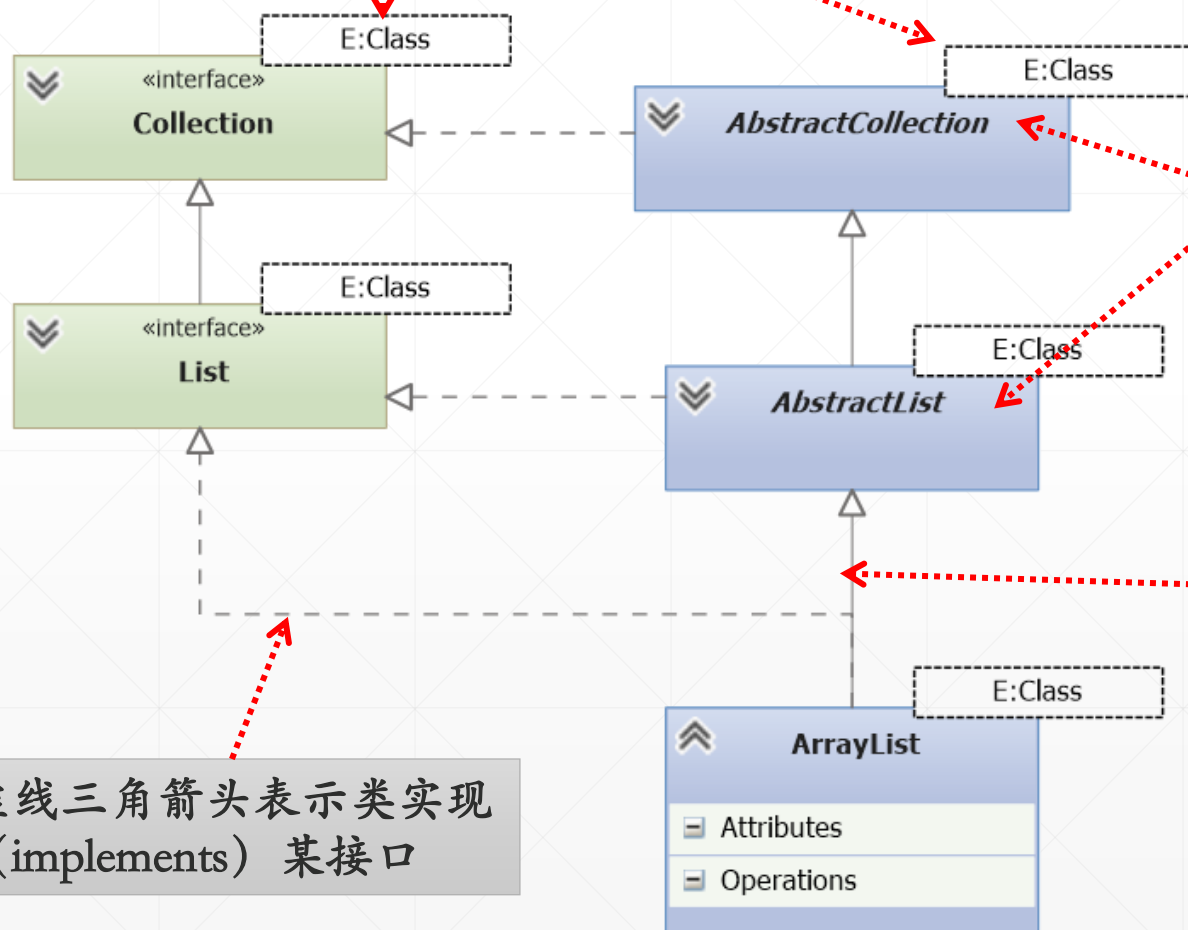
父类变量，可以引用子类对象。



接口变量，可以引用实现了这一接口的类的实例。

JDK中泛型类型间的关联实例

虚线框代表“泛型”，其中的字母代表“泛型参数”



“斜体”的类名表示这个类是一个抽象 (abstract) 类

实线三角箭头表示类之间的继承 (extends) 关系

虚线三角箭头表示类实现 (implements) 某接口

泛形多态示例

- 请对照着前页PPT所示之类型关联图，查看并运行本讲示例：

```
public class GenericPolymorphism {  
    public static void main(String[] args) {  
        List<Number> nums = new ArrayList<>();  
        nums.add(2);  
        nums.add(3.14);  
        for (Number number : nums) {  
            System.out.println(number.getClass().getName());  
        }  
    }  
}
```

GenericPolymorphism.java

示例中值得注意的地方：

1. ArrayList<>对象可以直接赋值给List<Number>变量
2. nums的add方法，能接收整数和浮点数

示例分析

- ✓ Collection泛型接口add()方法的定义如下:

```
interface Collection<E> {  
    public boolean add(E elt);  
    ...  
}
```

- ✓ ArrayList<E>实现的List<E>接口派生自Collection<E>, 所以可以写出以下代码:

```
//ArrayList<Number>可以被简写为ArrayList<>  
List<Number> nums = new ArrayList<Number>();  
nums.add(2);    //实际上是添加一个Integer对象, 自动装箱  
nums.add(3.14); //实际上是添加一个Double对象, 自动装箱
```



add()方法奏效的原因——Double和Integer均继承自抽象基类Number。

以下代码不是合法的“泛型”多态代码

```
// 定义一个类型为Integer的集合
List<Integer> ints = new ArrayList<Integer>();
ints.add(1); // 装箱
ints.add(2); // 装箱
// 因为Integer派生自Number，所以尝试着将List<Integer>
// 变量赋值给List<Number>变量
List<Number> nums = ints; // 出现编译时错误
```



List<Number>不是List<Integer>的基类型，两者之间没有“父子”关系，所以nums变量不能引用一个List<Integer>对象，但我们可以引入一个变通的方法解决这个问题，后面介绍。

有的时候，我们希望对泛型参数进行一些限制，比如，希望它实现XXX接口，必须是XXX的子类等等，这就引入了……

2 泛型约束



指定泛型类型参数的约束

- ✓ 指定泛型方法的泛型参数类型必须实现Comparable接口：

```
public static <T extends Comparable> T min(T[] a) {  
    //...  
}
```

- ✓ 需要时，可以给泛型参数指定多个约束条件：

```
T extends Comparable & Serializable
```



多个约束条件中最多只能有一个是类，并且它必须放在第一位。

类型参数约束示例

```
public class MyGenericClass<T extends Number> {  
  
    public static void main(String[] args) {  
  
        //由于Integer和Double都派生自Number  
        //所以，可以用它作为泛型参数，不会引发编译错误  
        MyGenericClass<Integer> ai = new MyGenericClass<>();  
        MyGenericClass<Double> ad = new MyGenericClass<>();  
  
        //下面代码将引起编译错误  
        //因为String不是Number的子类型。  
        //MyGenericClass<String> as = new MyGenericClass<>();  
  
    }  
}
```

MyGenericClass.java

泛型集合引发的麻烦.....

```
class Parent {  
    void working(){  
        System.out.println("Parent working");  
    }  
}  
  
class Child extends Parent {  
    @Override  
    void working(){  
        System.out.println("Child working");  
    }  
}
```

GenericWithWildcards.java

//创建两个泛型集合

```
List<Parent> parentList = new ArrayList<>();  
parentList.add(new Parent());  
//Child对象可以加入Parent集合中  
parentList.add(new Child());
```

```
List<Child> childList = new ArrayList<>();  
//Parent对象不能加放到Child集合中  
//childList.add(new Parent());  
childList.add(new Child());
```

父类集合变量无法引用子类集合对象:

```
//以下代码无法通过编译  
//parentList=childList;
```

解决父类集合变量引用子类集合对象的问题

```
//以下代码是正确的!  
//使用wildCard定义的集合, 可以引用任何一个泛型集合对象  
List<?> wildCardList = parentList;  
wildCardList = childList;  
wildCardList = new ArrayList<String>();  
//可以给“?”添加约束, 限制接收的集合对象必须是Parent的子类  
//现在, 这个变量就可以接收“父”与“子”类型的泛型集合类型了  
List<? extends Parent> classExtendsParentList = parentList;  
classExtendsParentList = childList;  
//以下这句代码将无法编译, 因为String没有派生自Parent, 不满足泛型约束  
//classExtendsParentList = new ArrayList<String>();
```

上述代码中的泛型参数“?”称为“**类型通配符 (wildcard types)**”，可以解决父类集合变量引用子类集合对象的问题。

Java中的“?”类型通配符约束



`List< ? extends T>`

要求“?”必须是T的子类。



`Collection<? super T> dest`

要求“?”必须是T的基类

类型通配符的应用

//使用wildcard, 统一处理子类 and 父类集合

```
static void doSomethingWithList(List<? extends Parent> parents){  
    for(var p:parents){  
        p.working();  
    }  
}
```

//使用wildcard, 输出任何一种类型的集合内容

```
static void printList(List<?> list) {  
    for(var item:list)  
        System.out.println(item);  
}
```

调用示例:

```
doSomethingWithList(parentList);  
doSomethingWithList(childList);  
  
printList(parentList);  
printList(childList);
```

JDK中“?”的真实示例

- ✓ JDK中定义了以下声明:

```
interface Collection<E> {  
    public boolean addAll(Collection< ? extends E > c);  
    ...  
}
```

- ✓ 所以以下代码都是对的，因为Integer和Double都派生自Number:

```
List<Number> nums = new ArrayList<Number>();  
List<Integer> ints = Arrays.asList(1, 2);  
List<Double> dbls = Arrays.asList(2.78, 3.14);  
nums.addAll(ints);  
nums.addAll(dbls);
```



注意：加入到nums集合中的是两个数组中的“单个”对象而不是集合对象ints和dbls本身。

动手动脑



编写一个泛型方法，能对任何一个数值类型的集合，找出它的最大元素。